

CSC 313: VDM-SL Quick Reference Cheat Sheet

Feature	Syntax	Short Example
Natural number	<code>nat</code>	<code>x : nat</code>
Token type	<code>token</code>	<code>UserId = token</code>
String	<code>seq of char</code>	<code>name : seq of char</code>
Enumerated type	<code><v1> <v2> ...</code>	<code>Status = <Active> <Blocked></code>
Set type	<code>set of T</code>	<code>stock : set of Book</code>
Map type	<code>map K to V</code>	<code>inventory : map ProductId to nat</code>
Record type	<code>R :: f1 : T1 ...</code>	<code>Order :: id : nat, total : nat</code>
Record constructor	<code>mk_R(v1, ...)</code>	<code>mk_Order(1, 250)</code>
State definition	<code>state S of ... inv mk_S(...) == ... init ... end</code>	(see Library example)
Invariant predicate	<code>inv mk_S(x,y) == ...</code>	<code>x inter y = {}</code>
Initialisation	<code>init mk_S(x,y) == x={} and y={}</code>	
External variable access	<code>ext rd / wr var : type</code>	<code>ext wr stock : set of Book</code>



Pre-condition	<code>pre ...</code>	<code>pre b not in stock~</code>
Post-condition	<code>post ...</code>	<code>post stock = stock~ union {b}</code>
Assignment (in post)	<code>:=</code> (not used inside post; use equality)	
Old (pre-state) value	<code>var~</code>	<code>stock~</code>
Result variable	<code>RESULT</code>	<code>post RESULT = b in stock~</code>
Set union / intersection / difference	<code>union inter \</code>	<code>s1 union s2</code>
Set membership	<code>in set</code>	<code>x in set s</code>
Subset	<code>subset</code>	<code>s1 subset s2</code>
Map domain / range	<code>dom m rng m</code>	<code>dom menu</code>
Map override (update/merge)	<code>++</code>	<code>m1 ++ {k -> v}</code>
Empty map	<code>{ ->}</code>	<code>cart = { ->}</code>
Universal quantification	<code>forall x in set s & P(x)</code>	<code>forall b in set borrowed & b in set stock</code>
Sum expression	<code>sum x in set s & e</code>	<code>sum i in set items & price * qty</code>



**SCHOOL OF
SCIENCE AND
TECHNOLOGY**
PAN-ATLANTIC UNIVERSITY

Record field update

`mu(rec, field :=
newVal)`

`mu(p, stock := p.stock
- 1)`

Cardinality

`card S`

`card stock(number of
elements)`
